

Lab 19: Exceptions and Intro to OOP

- Due Thursday by 11:59pm
- Points 0.8
- Submitting an external tool
- Available Jul 14 at 6pm - Jul 16 at 11:59pm

Learning Objectives

- Catch exceptions from other functions
- Writing Basic Objects
- Adding Methods to Objects

Completion Requirements

- Submit Lab19.py

Q1: SSN Validation

Write a function, **ssnValidation (ssn)** , that validates the given SSN number based on the following rules.

1. The string entered must be either 9 characters or 11 characters. Raise a ValueError exception with description "Invalid ssn length".
2. If the input is 9 characters then check that all characters are digits. Raise a ValueError exception with description "Invalid character in 9 digit format".
3. If the input is 11 characters then check that all characters except characters at index 3 and 6 are digits. The characters at indices 3 and 6 must be '-'. Raise ValueError with description "Invalid character in 11 character format."
4. Finally, return the 9 character format with all digits as validated SSN. If 11 character format was given then it must be converted to 9-digit format.



Examples:

```
>> print(ssnValidation("0000000000"))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: Invalid ssn length
>>
>> print(ssnValidation("000000000A"))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
```

```

ValueError: Invalid character found in 9 digit format
>> print(ssnValidation("000-00-000A"))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: Invalid character found in 11 digit format
>>
>> print(ssnValidation("000-00-0000"))
0000000000
>>
>> print(ssnValidation("0000000000"))
0000000000

```

Q2: Toys

Write a class `Toy` with the following six instance methods:

1. **`setName(self, name)`**: Sets attributes for toy name.
2. **`setRecommendedAge(self, lowerLimit, upperLimit = 99)`**: Sets attributes that remember the lower and upper age limits (inclusive limits).
3. **`setPrice(self, price)`**: Sets the price attribute.
4. **`isRecommended(self, age)`**: returns True if the given age is in the recommended limit. False otherwise.
5. **`isInBudget(self, budget)`**: return True if the price is less than budget. False otherwise.
6. **`toString(self)`**: Creates a string with information from a toy. The price of the toy is rounded to 2 dp. See example.

Note that we have not given you any attribute names. What attributes you save, and their names is your decision. This is an example of abstracting away from the details. As long as the 6 functions given above produce consistent results, the user does not need to know the internal details.

Example 1:

```

>> t = Toy()
>> t.setName("Kiddieland Plane Activity Ride-On")
>> t.setRecommendedAge(1, 5)
>> t.setPrice(35.4999)
>> t.toString()
Kiddieland Plane Activity Ride-On is recommended for children 1 to 5 years ol
d and costs $35.50
>> t.isRecommended(6.5)
False
>> t.isInBudget(20)
False

```

Q3: Dictionary

This question show you how the dictionary can be built on top of lists in python. Write a class `Dictionary` with the following attributes and instance methods:

Attributes

1. `lst`: This should be a list initialized to 100 `None` values in the constructor. The length of the list will be 100.

Methods

1. `set(self, key, value)`
`key` will be a string. Loop over the string and use `ord` method to get integer equivalent for all characters. Sum them up to create an integer representation of the string. This number can be larger than 100. Use the modulo operation to calculate the remainder of this integer with 100. Now you will have a number that is a valid index for the list. Store the tuple `(key, value)` at the calculated list index.
2. `get(self, key)`
Using the process described in the set method, calculate the index for the `key`. Retrieve the tuple from the list and return the value part of the tuple. If you find `None` instead of a tuple then raise `KeyError` Exception with the message "<key> not found in Dictionary" where <key> should be the key supplied by the user.

Note: This is not a complete dictionary implementation because it does not have any mechanism to check if two keys have the same index mapping to list. It also does not support all kinds of keys.

Example (Normal Cases):

```
>> my_dict = Dictionary()
>> my_dict.set("A", 100)
>> my_dict.get("A")
100
>> my_dict.set("cat", "animal")
>> my_dict.get("cat")
'animal'
>> my_dict.set("apple", 5.50)
>> my_dict.get("apple")
5.5
```

Example (Exception Cases):

```
>> my_dict = Dictionary()  
>> my_dict.set("cat", "animal")  
>> my_dict.get("dog")  
KeyError: dog not found in Dictionary  
>> my_dict.get("A")  
KeyError: A not found in Dictionary
```

This tool needs to be loaded in a new browser window

Load Lab 19: Exceptions and Intro to OOP in a new window

